Docker And Kubernetes

Shubham Dalvi

Agenda

Agenda

- Introduction and Overview
- Containers
- Docker
- Docker Swarm
- Kubernetes
- Demos and Practices

My Introduction

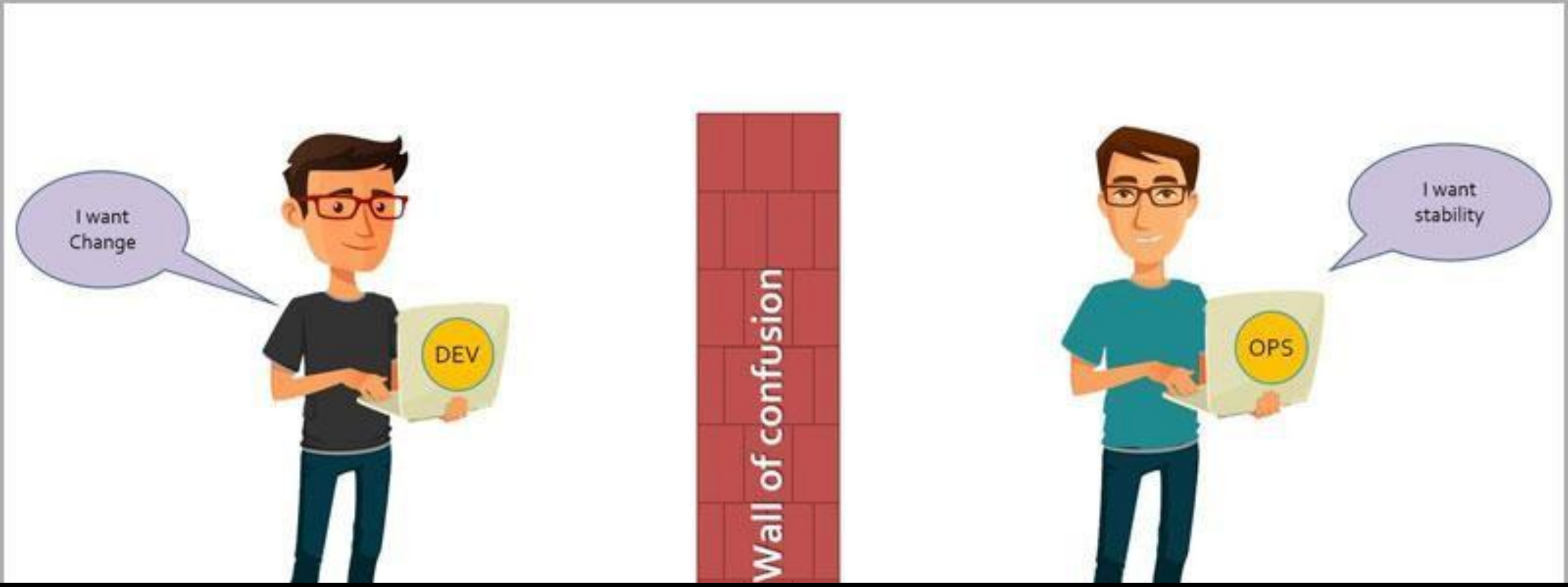
Shubham Dalvi

- AWS Authorized Champion Instructor
- Google Authorized Instructor
- Corporate Trainer and Consultant
- RPS Consulting

Specialty

- Cloud Computing
- Infrastructure services

Introduction

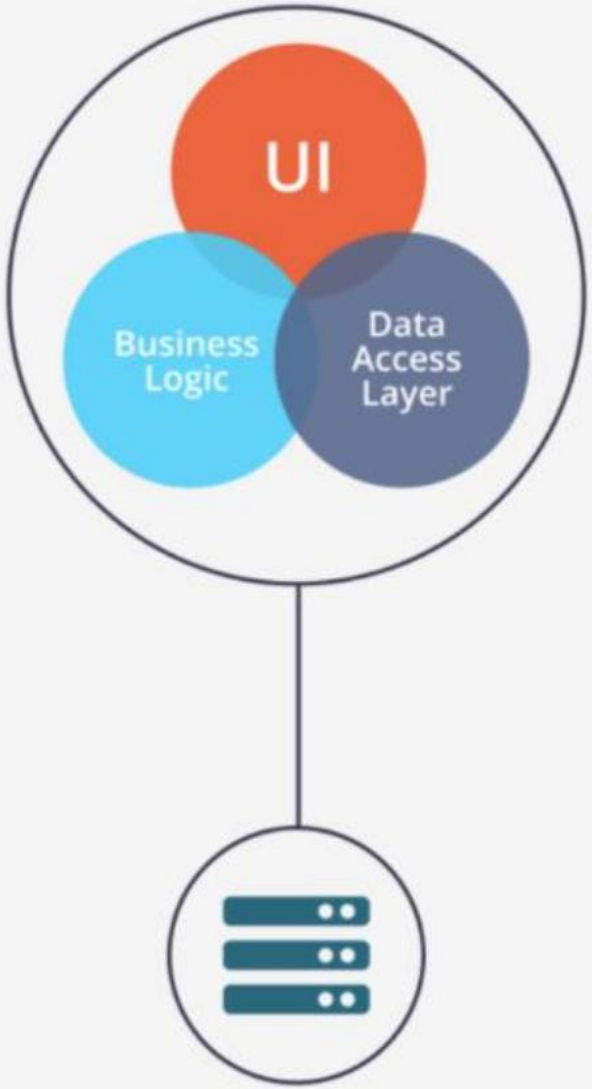


Development vs Operations

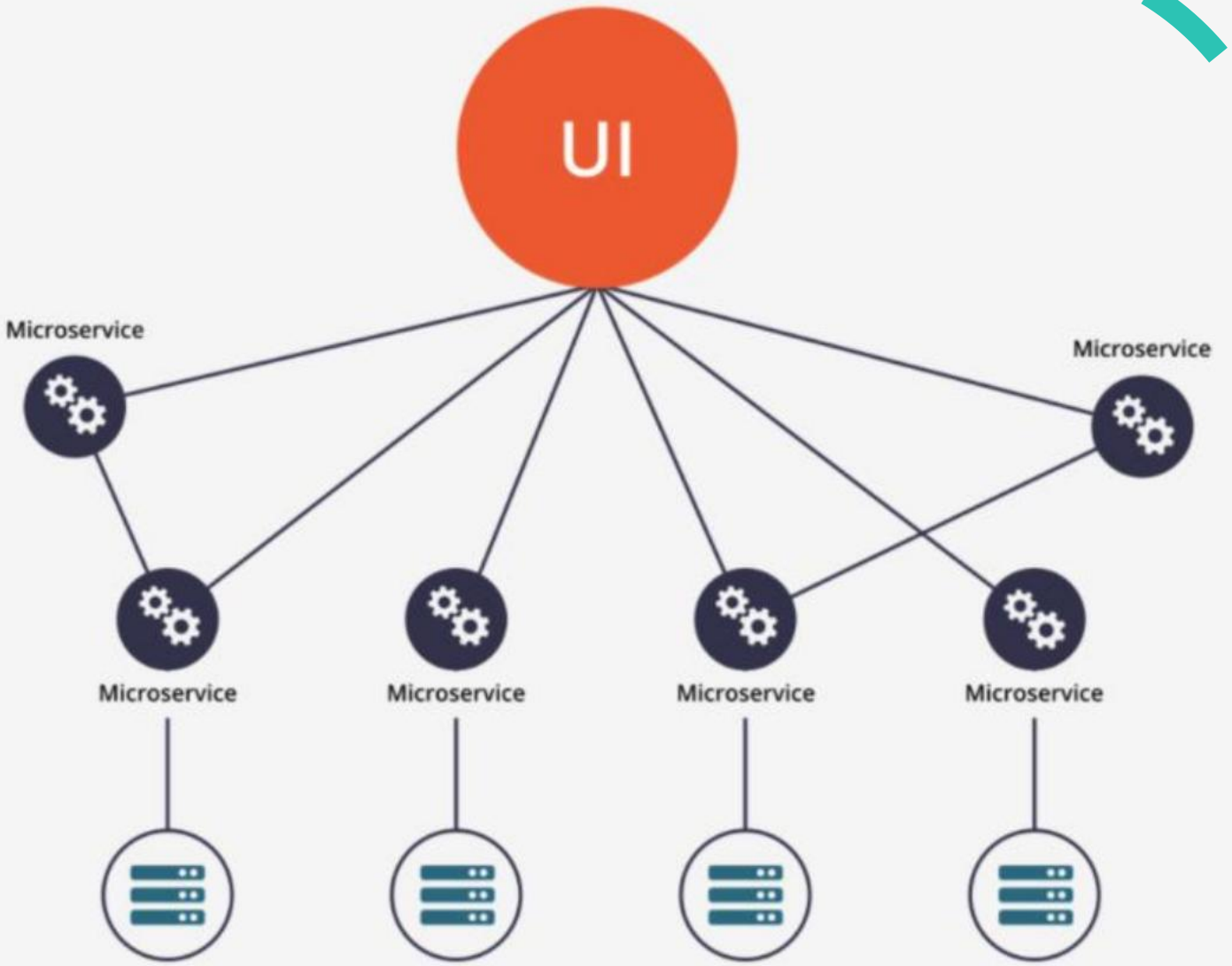
My code on my machine

vs.

My code on production



Monolithic Architecture



Microservice Architecture

Containerization

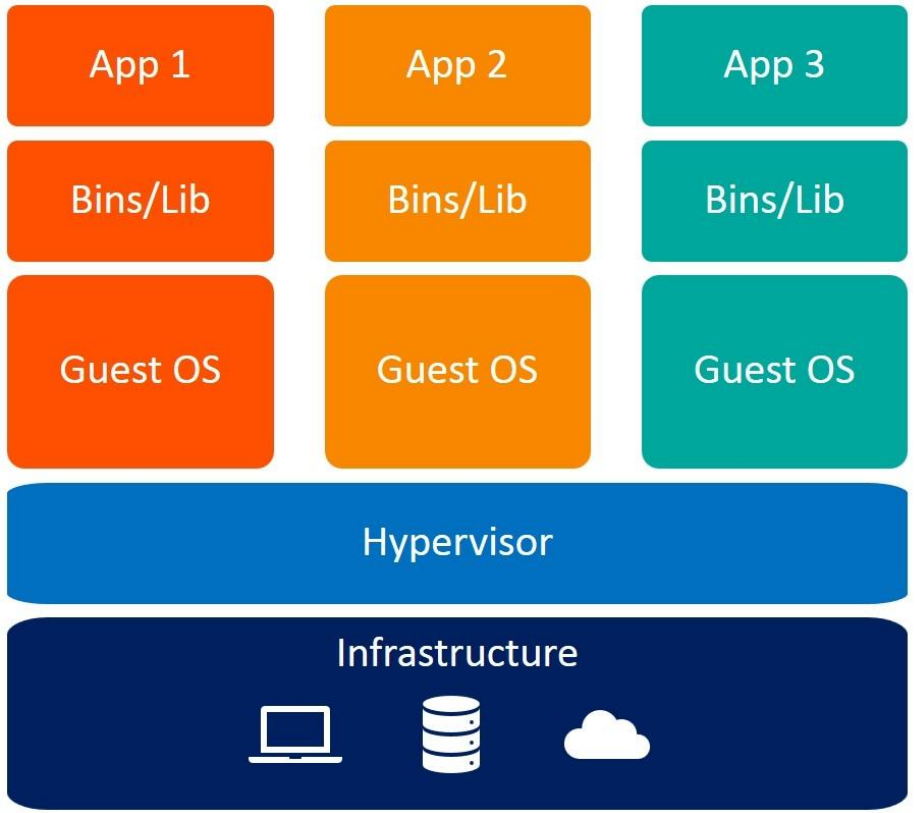
Application Hosting Journey

- Physical Machine
- Virtualization
- Containers
- Cloud

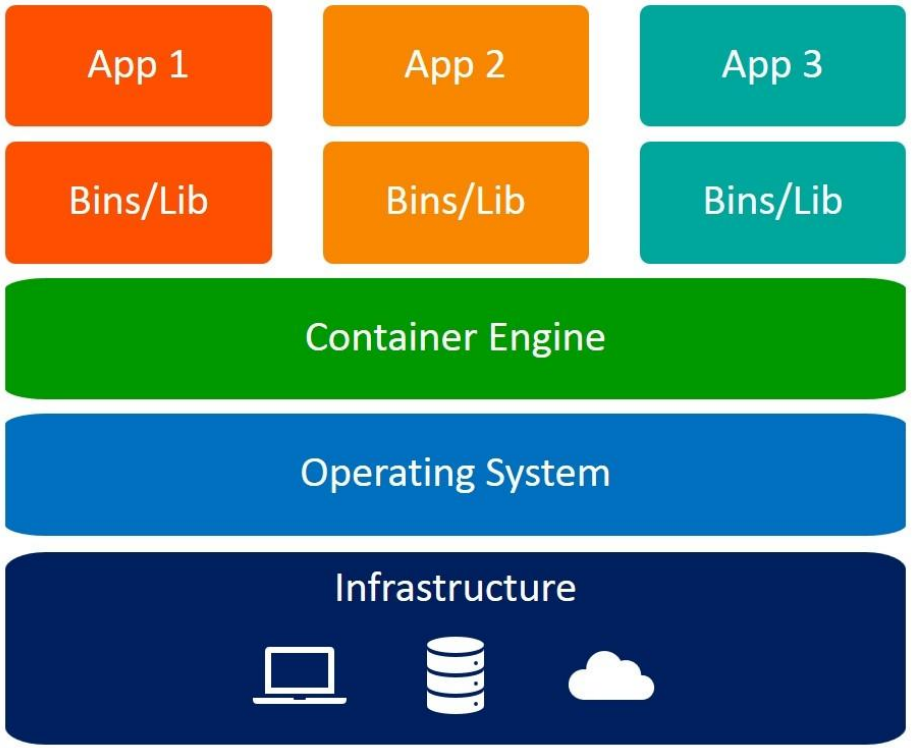
Container Definition

- A container is like a lightweight and portable box that holds everything needed for a piece of software to run.
- It includes the software code, its dependencies, and settings, making it easy to move the software between different computing environments, like from a developer's laptop to a testing server or to the cloud.
- Containers help ensure that the software works consistently no matter where it's run.

Container vs VM



Virtual Machines

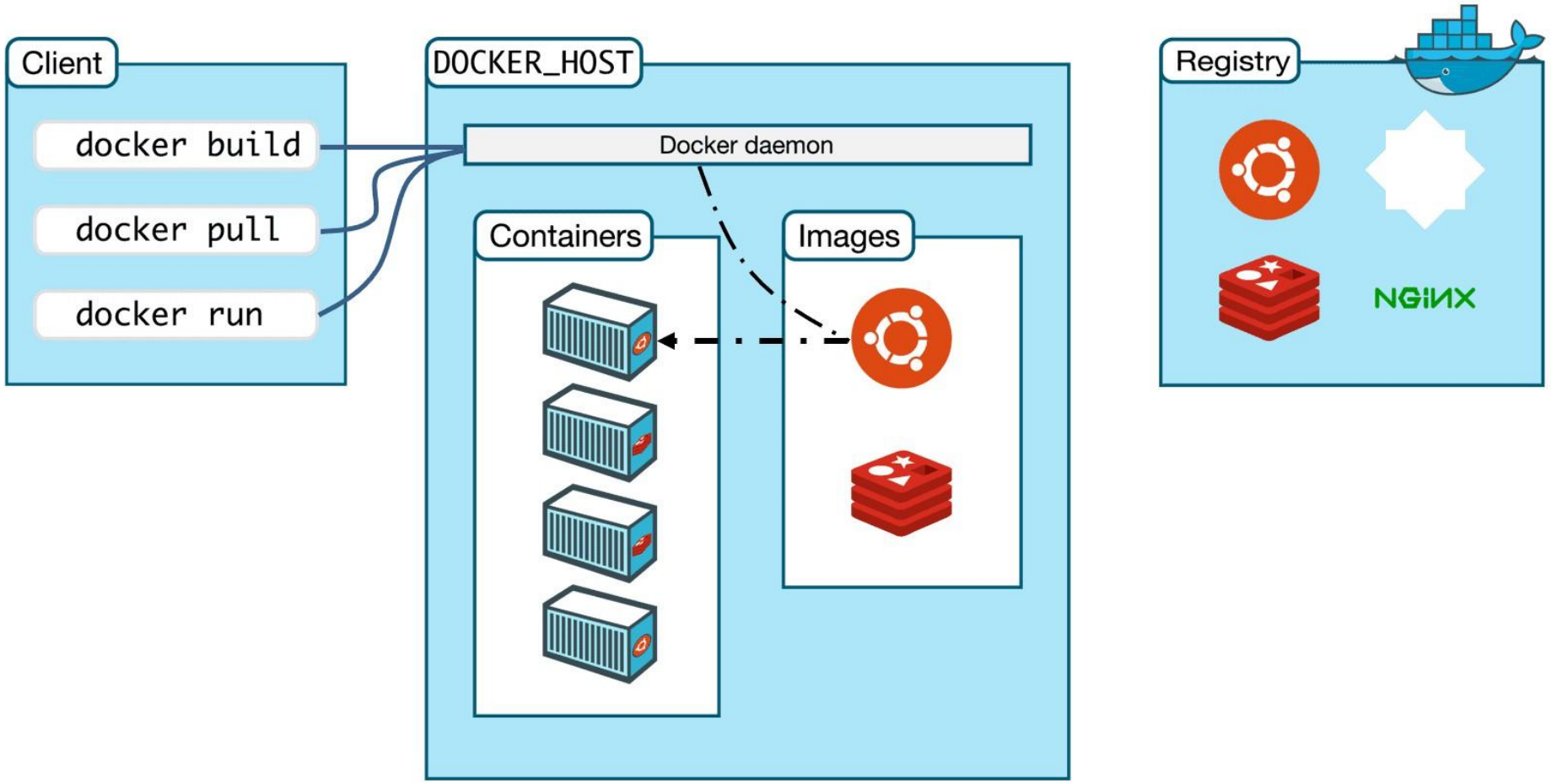


Containers

Docker

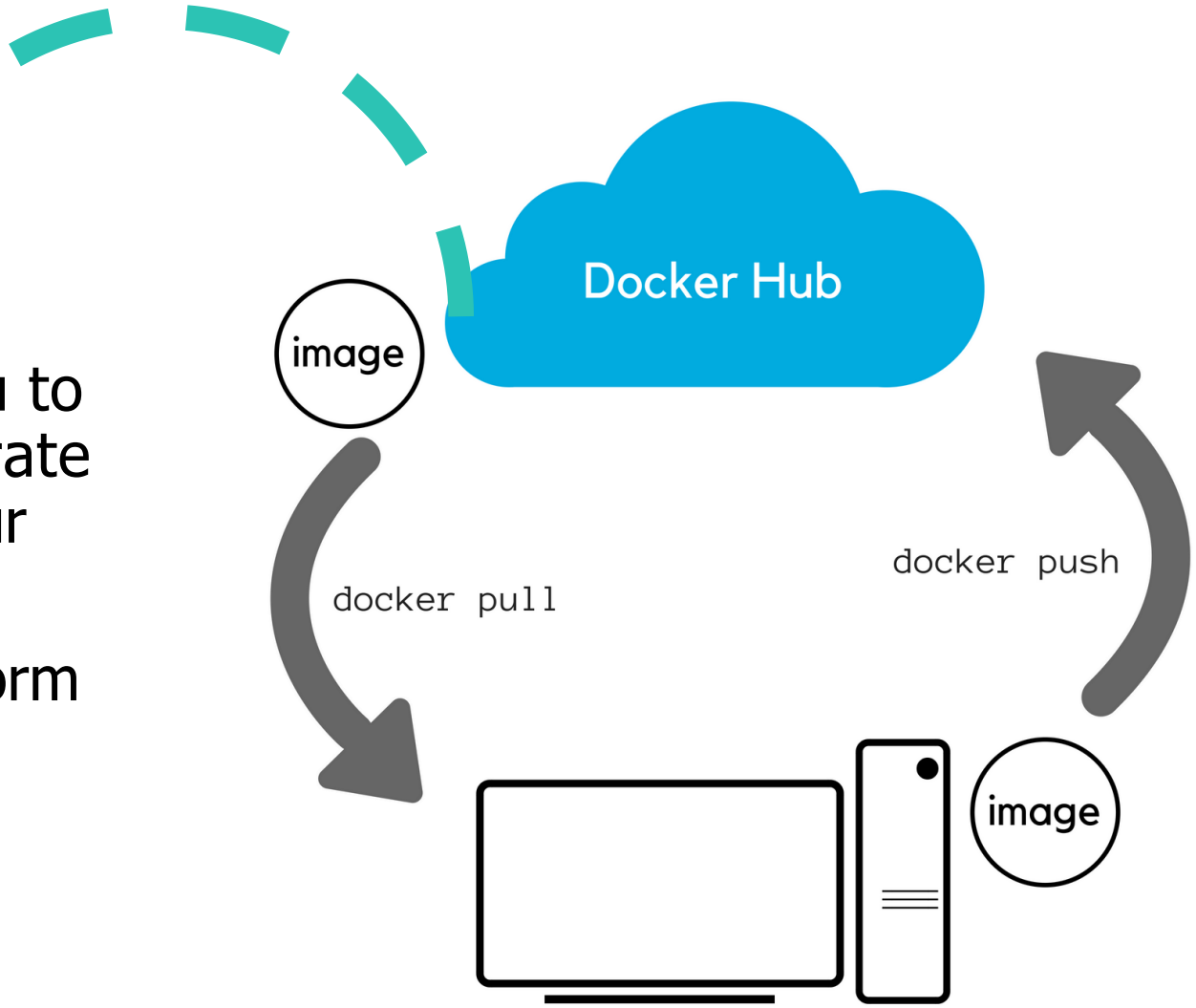
- Docker is a containerization platform that allows you to package an application and its dependencies into a standardized unit called a container.
- Other similar applications:
  - Podman
  - Containerd
  - LXC
  - OpenShift
  - Etc.

Docker Architecture



Docker Hub

- Docker Hub is a cloud-based registry service that allows you to share Docker images, collaborate with others, and automate your image deployment.
- It serves as a centralized platform for managing Docker images.

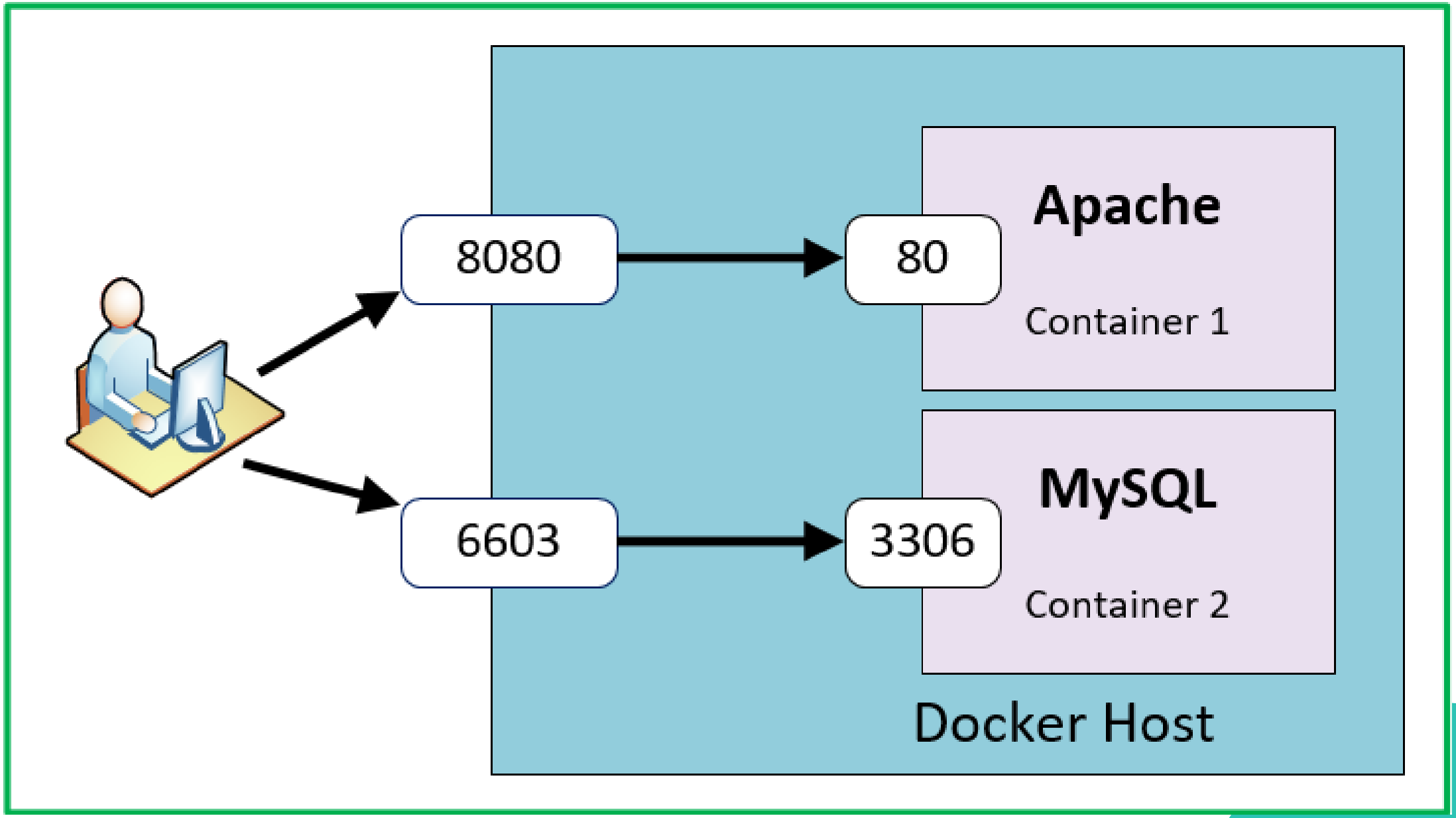


Demo 1

- Install Docker
- Create containers
- Manage containers

Port Forwarding

- Port forwarding is a networking technique that allows traffic to pass through a network's firewall, enabling external devices to access specific services on a local network.
- Port forwarding involves redirecting communication requests from one IP address and port number to another. It's used to expose services, like web servers or gaming consoles, to the internet.

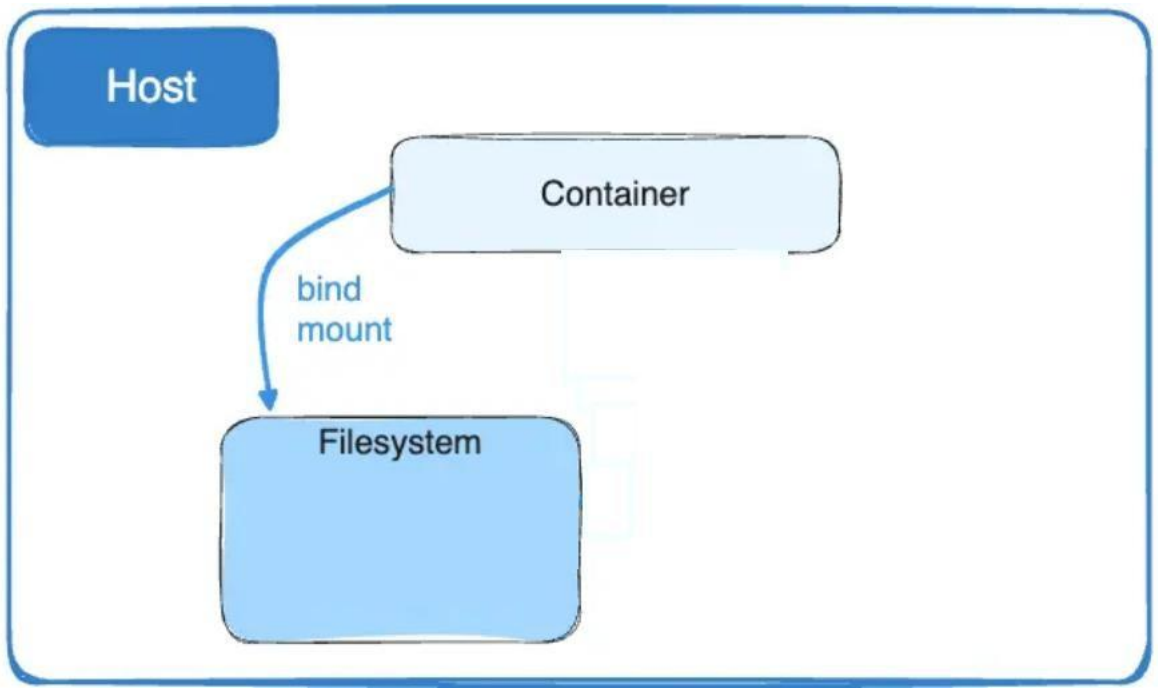

8080
80
Apache
Container 1
6603
3306
MySQL
Container 2
Docker Host

Environment Variable

- Docker environment variables are key-value pairs that store data used by processes within a Docker container.
- These variables are commonly employed to configure applications and services running inside the container.

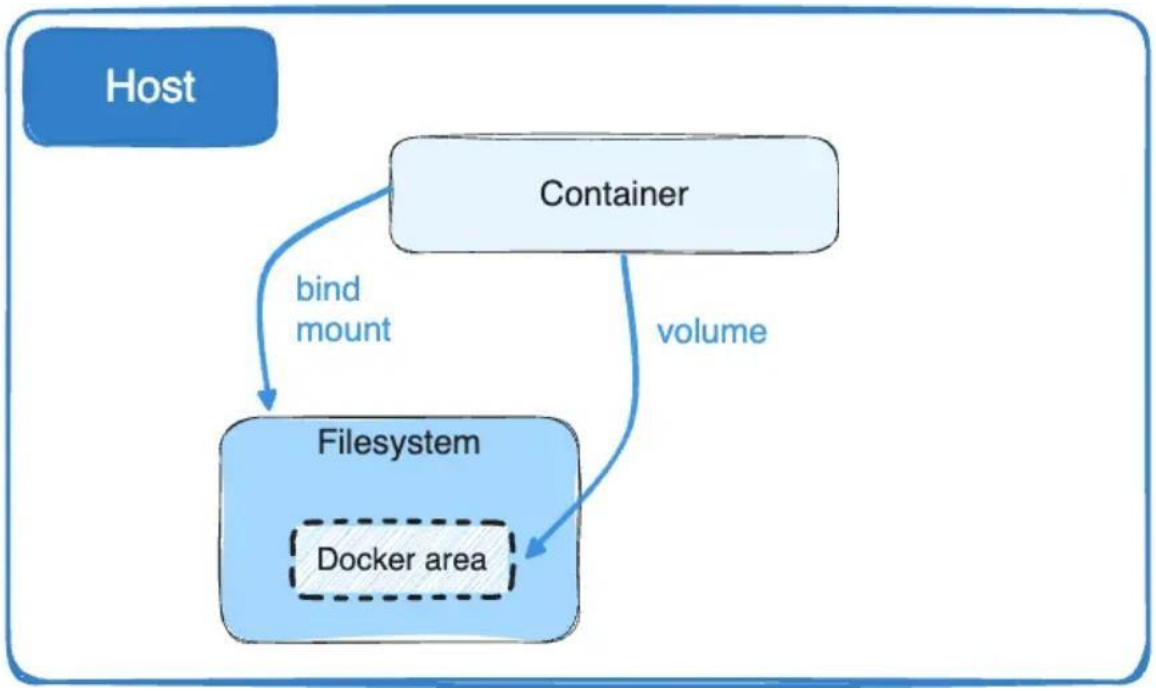
Volumes

Bind Mount



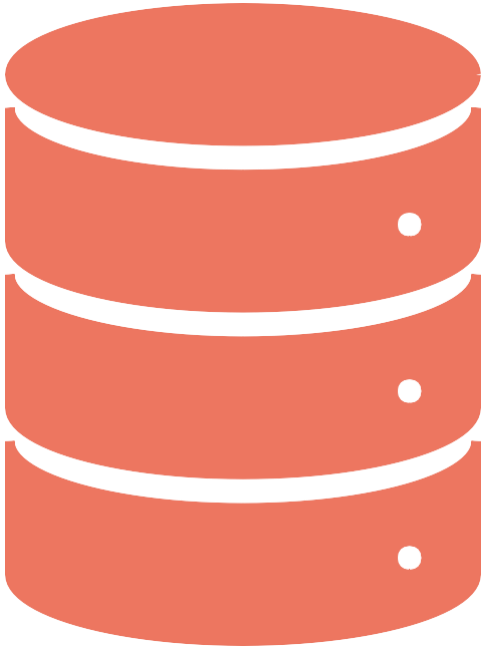
- A bind mount is a type of Docker mount that lets you share a directory from the host's filesystem into the container.
- This is useful for sharing data between the host and the container, or for mounting a volume into a container.
- To use a bind mount, you need to specify the source and target directories.
- Bind mounts have been around since the early days of Docker. Bind mounts have limited functionality compared with volumes.

Volumes



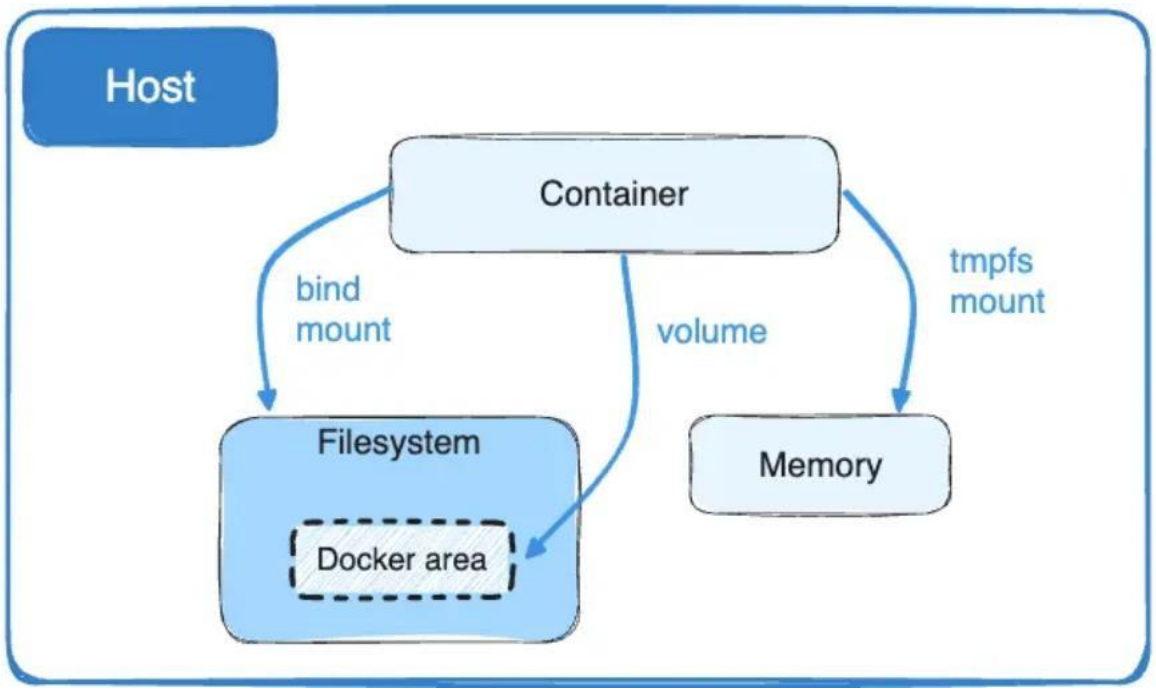
- Volumes are the preferred mechanism for persisting data generated by and used by Docker containers.
- While bind mounts are dependent on the directory structure and OS of the host machine, volumes are completely managed by Docker.

Volumes Features



- Volumes are easier to back up or migrate than bind mounts.
- You can manage volumes using Docker CLI commands or the Docker API.
- Volumes work on both Linux and Windows containers.
- Volumes can be more safely shared among multiple containers.
- Volume drivers let you store volumes on remote hosts or cloud providers, to encrypt the contents of volumes, or to add other functionality.

Tmpfs Mount



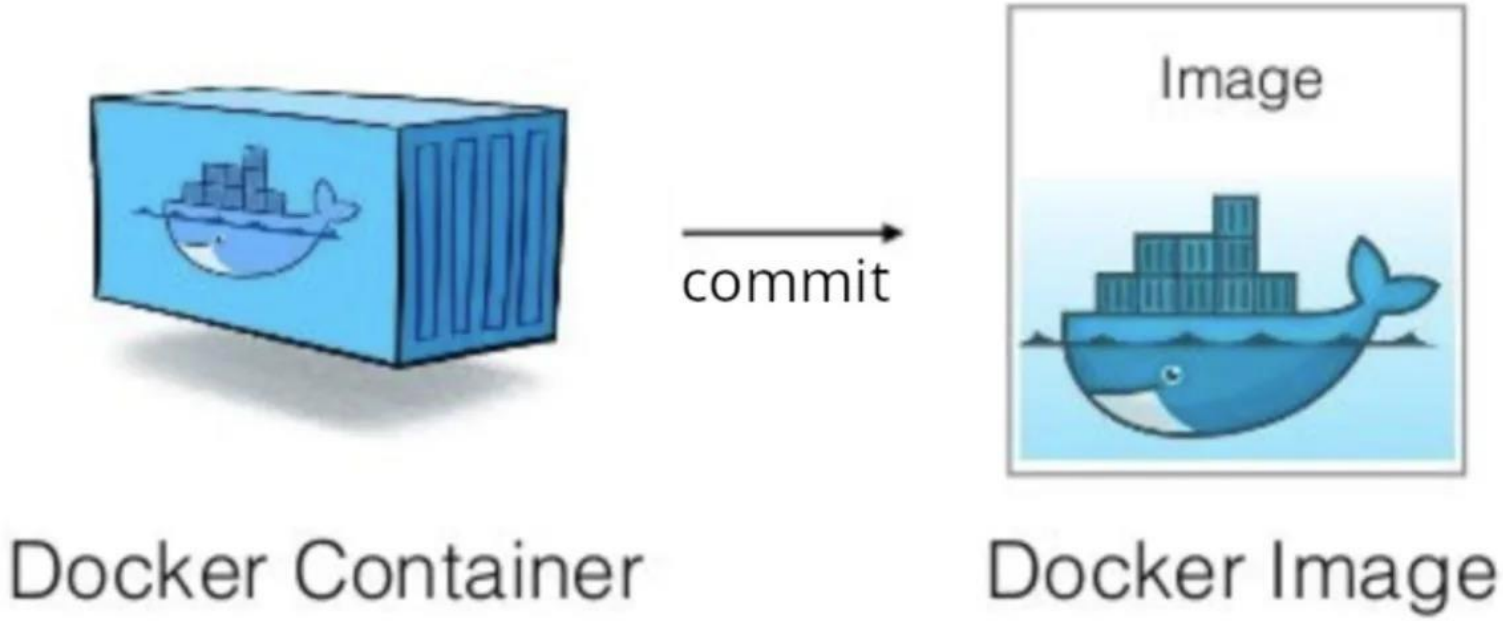
- A tmpfs mount is a temporary file system that exists in memory and/or swap partitions.
- It's different from volumes and bind mounts, which are persistent. When a container stops, the tmpfs mount is removed, and any files written there are lost.
- Mounting directories as tmpfs can speed up access to their files or ensure that their contents are automatically cleared upon reboot.

Custom Images

Docker Commit

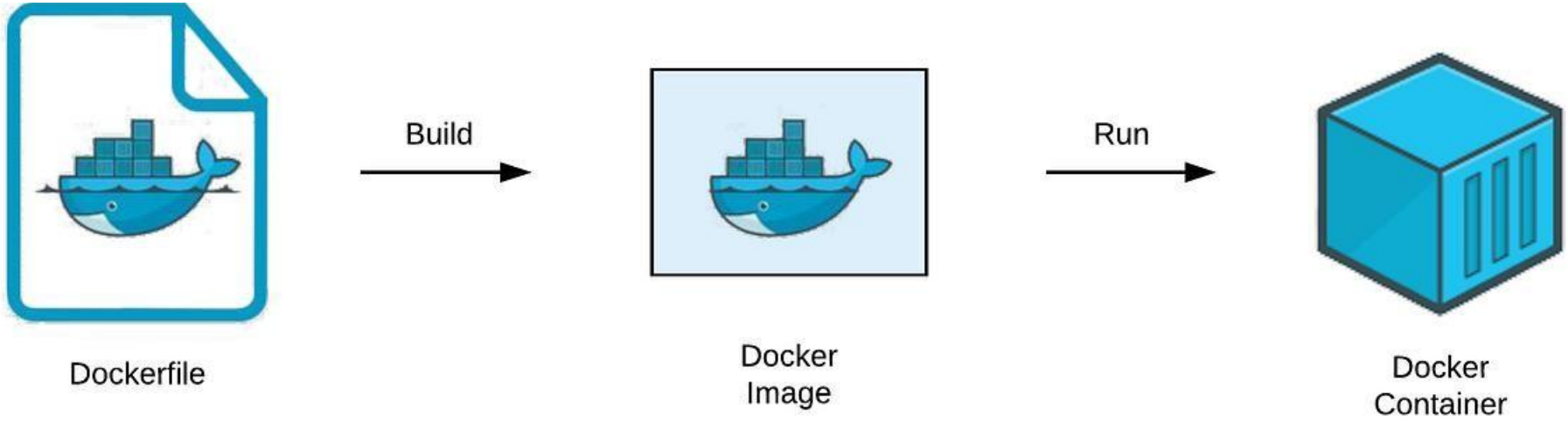
The docker commit command saves changes made to a running Docker container and creates a new image from it.

This allows you to create custom images based on existing ones, or to save changes made to a container for later use.



Docker File

- Dockerfile is a plaintext configuration file that contains a set of instructions used to build a Docker image.
- It serves as a blueprint for creating a reproducible and consistent environment.



Sample Docker File

```
FROM node:14
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
EXPOSE 3000
CMD [ "npm", "start" ]
```

Docker File Instructions

Instruction	Description
FROM	Specifies the base image for your Docker image. It's the starting point for the build process.
LABEL	Adds metadata to the image. Commonly used for specifying the maintainer, version, and other information about the image.
RUN	Executes commands in a new layer on top of the current image and commits the results. Used to install packages, update the system, or perform any setup required for the image.
WORKDIR	Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD instructions that follow it. It affects the context of these instructions.
COPY	Copies files or directories from the host machine to the container. Used to transfer application code, configuration files, and other necessary files into the image.

Docker File Instructions Contd.

Instruction	Description
ADD	Similar to COPY, but with additional features such as remote URL support and automatic tar extraction. While COPY is preferred for most use cases, ADD can be useful for certain scenarios.
EXPOSE	Informs Docker that the container will listen on specified network ports at runtime. It doesn't actually publish the ports; it's more of a documentation feature to help users understand which ports to use.
ENV	Sets environment variables in the image. These variables can be accessed during the build process and used by processes running in the container.
CMD	Provides default arguments for the ENTRYPOINT instruction or specifies the command to run when the container starts. It can be overridden by passing a command when running the container.
ENTRYPOINT	Configures the container to run as an executable. It allows you to set the main command that will be executed when the container starts. It can also be overridden by passing a command when running the container.

Hosting Stack

Docker Compose

- Compose is a tool for defining and running multi-container Docker applications.
- With Compose, you use a YAML file to configure your application's services.
- Then, with a single command, you create and start all the services from your configuration.

Docker Compose Contd.

- Compose works in all environments; production, staging, development, testing, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:
  - Start, stop, and rebuild services
  - View the status of running services
  - Stream the log output of running services
  - Run a one-off command on a service

Docker Networking

Networking Overview

- Container networking refers to the ability for containers to connect to and communicate with each other, or to non-Docker workloads.
- Containers have networking enabled by default, and they can make outgoing connections. A container has no information about what kind of network it's attached to, or whether their peers are also Docker workloads or not.
- A container only sees a network interface with an IP address, a gateway, a routing table, DNS services, and other networking details.

User-defined Networks

- You can create custom, user-defined networks, and connect multiple containers to the same network.
- Once connected to a user-defined network, containers can communicate with each other using container IP addresses or container names.

```
$ docker network create -d bridge my-net
$ docker run - network=my-net -itd - name=container3
busybox
```

Network Drivers

Driver	Description
bridge	The default network driver.
host	Remove network isolation between the container and the Docker host.
none	Completely isolate a container from the host and other containers.
overlay	Overlay networks connect multiple Docker daemons together.

Docker Troubleshooting

CLI output formatting

- Container commands are clipped once they exceed a certain length in the terminal output. That prevents you from inspecting the command in its entirety.
- You can read and compare any parameters in full. Nothing is hidden.

```
$ docker container ls - all - format ‘{{ json . }}’ |
python3 -m json.tool - json-lines
```

Container Logging

- By following best practices, any active application running within a Docker container will produce log outputs.
- While you might view logging as a problem-catching mechanism, many running containers don’t experience issues.

```
$ docker logs [container ID]
$ docker logs - tail 100 [container ID]
```

Docker Log Options

Option	Short	Default	Description
--details			Show extra details provided to logs
--follow	-f		Follow log output
--since			Show logs since timestamp (e.g. 2013-01-02T13:23:37Z) or relative (e.g. 42m for 42 minutes)
--tail	-n	all	Number of lines to show from the end of the logs
--timestamps	-t		Show timestamps
--until			API 1.35+ Show logs before a timestamp (e.g. 2013-01-02T13:23:37Z) or relative (e.g. 42m for 42 minutes)

Container Orchestration

Need of Container Orchestration

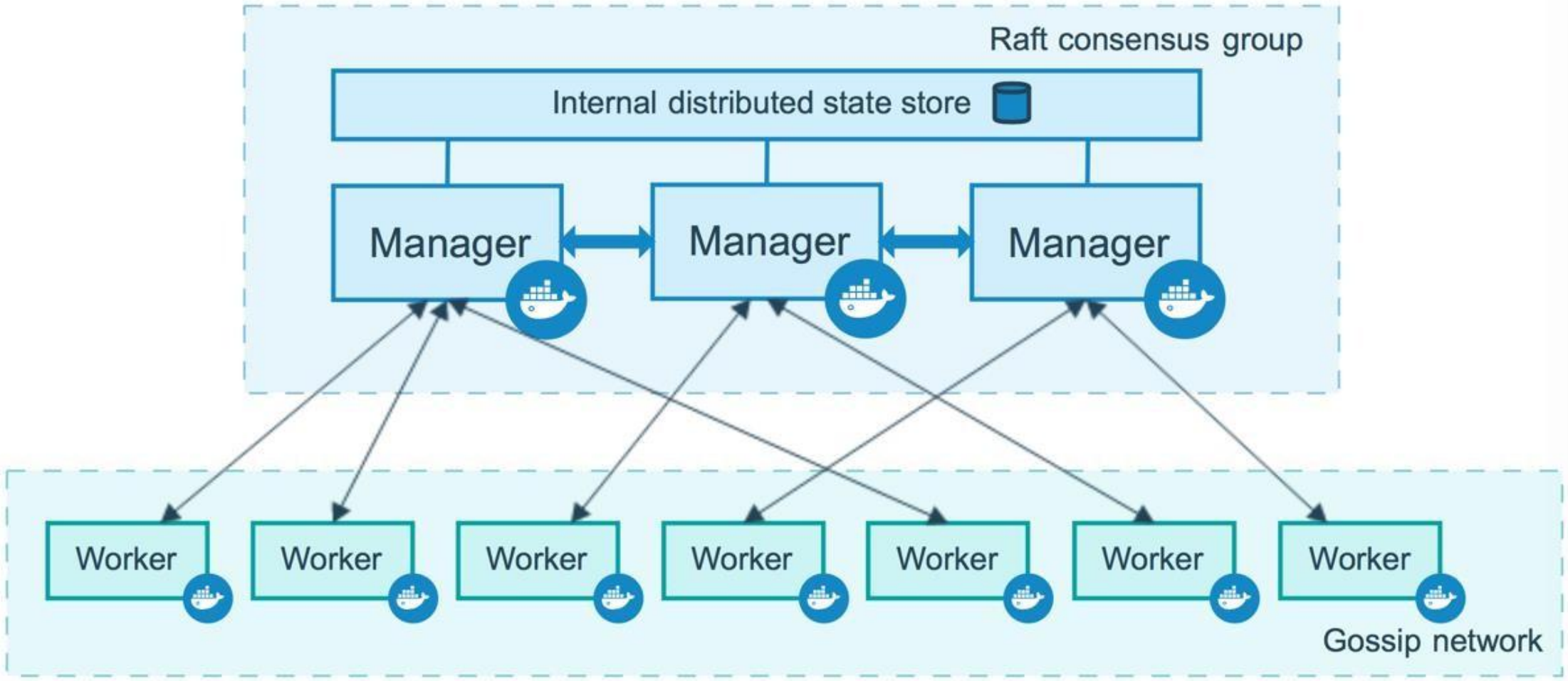
- Container orchestration is a crucial aspect of managing and scaling containerized applications effectively.
- As the adoption of container technologies like Docker has increased, the need for orchestration solutions has become more apparent.

List of Container Orchestration applications

- Kubernetes
- Docker Swarm
- Red Hat OpenShift
- Nomad
- Mesosphere DC/OS

Docker Swarm

- Swarm provides many tools for scaling, networking, securing and maintaining your containerized applications, above and beyond the abilities of containers themselves.
- Swarm never creates individual containers. Instead, all Swarm workloads are scheduled as services, which are scalable groups of containers with added networking features maintained automatically by Swarm.


Raft consensus group
Internal distributed state store
Manager
Manager
Manager
Worker
Worker
Worker
Worker
Worker
Worker
Worker
Gossip network

Docker Key Concepts: Nodes

- A node is an instance of the Docker engine participating in the swarm.
- You can also think of this as a Docker node. You can run one or more nodes on a single physical computer or cloud server, but production swarm deployments typically include Docker nodes distributed across multiple physical and cloud machines.

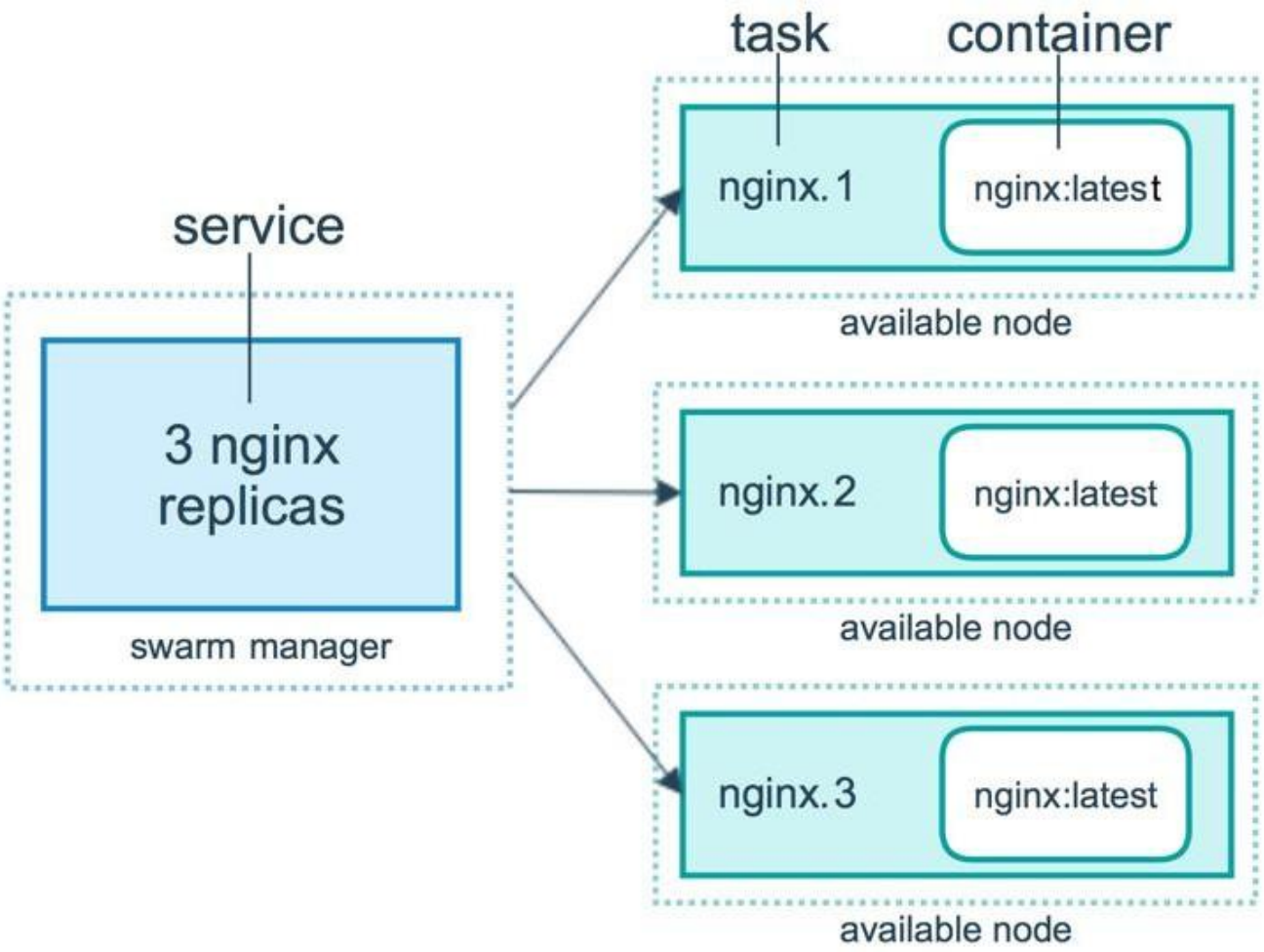
Docker Key Concepts: Services and tasks

- A service is the definition of the tasks to execute on the manager or worker nodes. It is the central structure of the swarm system and the primary root of user interaction with the swarm.
- In the replicated services model, the swarm manager distributes a specific number of replica tasks among the nodes based upon the scale you set in the desired state.
- A task carries a Docker container and the commands to run inside the container.

Docker Key Concepts: Load balancing

- The swarm manager uses ingress load balancing to expose the services you want to make available externally to the swarm.
- The swarm manager can automatically assign the service a published port or you can configure a published port for the service.
- External components, such as cloud load balancers, can access the service on the published port of any node in the cluster whether or not the node is currently running the task for the service.

Docker Key Concepts



Kubernetes

- Kubernetes, often abbreviated as K8s, is a powerful and popular open-source container orchestration platform for automating the deployment, scaling, and management of containerized applications.
- It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF).
- Kubernetes provides a robust and flexible framework for container orchestration, making it easier to manage and scale applications in complex, dynamic environments.

Kubernetes vs Docker Swarm

Attribute	Docker Swarm	Kubernetes
Simplicity and Ease of Setup	Easier to set up and use. Ideal for simpler applications or for those new to container orchestration. Native integration with Docker CLI and Docker Compose files.	More complex to set up and manage, suitable for larger and more complex environments. Steeper learning curve but offers more control and flexibility.
Lightweight	More lightweight with fewer components, leading to simpler operations.	Relatively heavier, with more components and larger infrastructure footprint.
Functionality	Capable but with limited features compared to Kubernetes. Less suited for complex orchestration, scaling, and self-healing.	Feature-rich, offering advanced capabilities like rollbacks, rollouts, autoscaling, and self-healing. Suitable for complex orchestration needs.

Kubernetes vs Docker Swarm

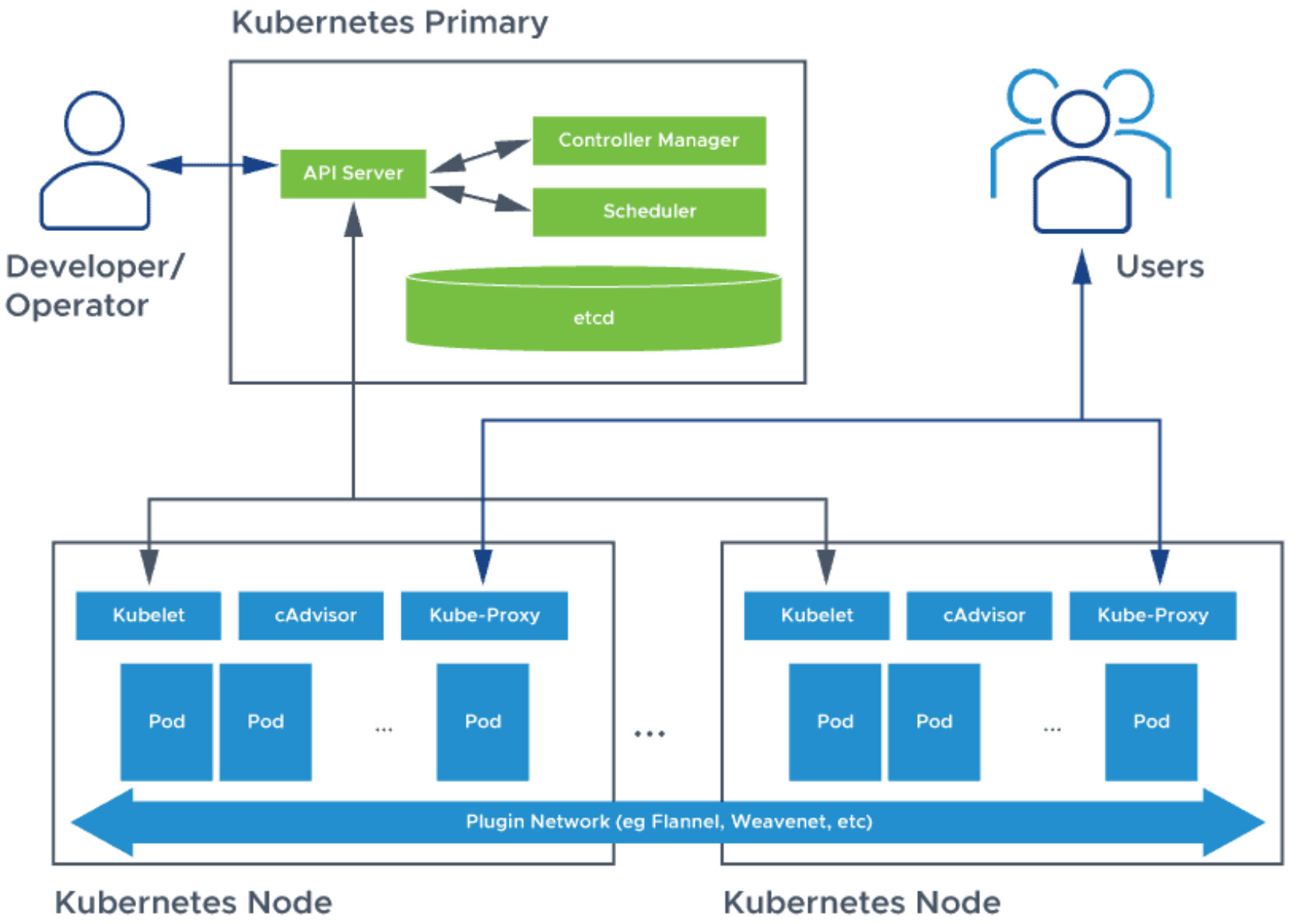
Attribute	Docker Swarm	Kubernetes
Integration	Tight integration with Docker, advantageous for Docker-centric workflows.	Broad integration capabilities with various cloud platforms and technologies.
Scalability and Flexibility	Less scalable and flexible compared to Kubernetes. Suitable for smaller-scale deployments.	Highly scalable and flexible, capable of handling complex and large-scale deployments efficiently.
Ecosystem and Community	Smaller ecosystem and community compared to Kubernetes.	Large ecosystem and extensive community support. Widely supported by cloud services like Amazon EKS, Google GKE, and Microsoft AKS.

Use Cases

- Docker Swarm: Ideal for smaller-scale applications, startups, or for those looking for simplicity and ease of use without the need for advanced orchestration features.
- Kubernetes: More suitable for larger organizations and complex applications that require the scalability, flexibility, and advanced features that Kubernetes offers.

Importance in Container Orchestration

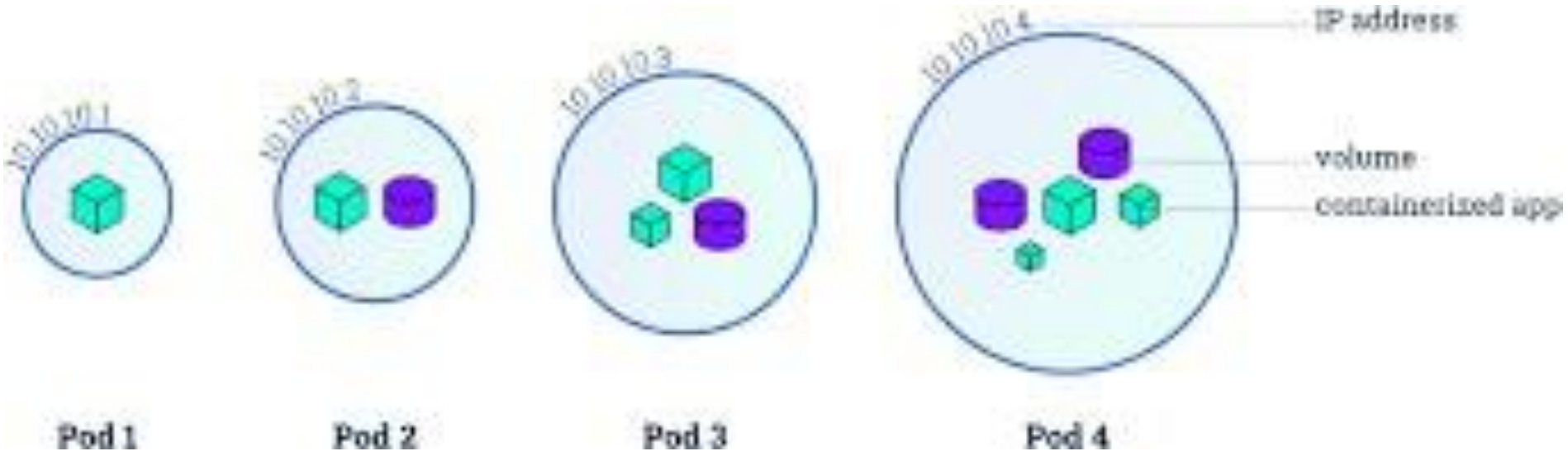
- Efficient Container Management
- Scalability
- Automation
- Portability
- Resilience
- Resource Optimization


Kubernetes Primary
Developer/
Operator
API Server
Controller Manager
Scheduler
etcd
Users
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
...
Kubelet
cAdvisor
Kube-Proxy
Pod
Pod
...
Pod
Plugin Network (eg Flannel, Weavenet, etc)
Kubernetes Node
Kubernetes Node

Kubernetes Pods

- Pod is the smallest deployable unit and represents a single instance of a running process in a cluster.
- A Pod can contain one or more tightly coupled containers, which share the same network namespace, storage, and other resources.

Kubernetes Pods Contd.



Kubernetes Concepts

- Replication Controllers and ReplicaSets
- Services
- Deployments
- ConfigMaps and Secrets
- Persistent Volumes and Persistent Volume Claims
- Namespaces

Init container

- Init containers are a special type of container in Kubernetes that run before the main application container. They are used to initialize new Pods and ensure the environment is set up correctly for the main containers to run successfully.
- Init containers are similar to regular containers, but they run to completion and run before the main container starts. They can contain utilities or setup scripts that are not present in an app image.

Init container Contd.

- Init containers can contain utilities or custom code for setup that are not present in an app image. For example, there is no need to make an image FROM another image just to use a tool like sed, awk, python, or dig during setup.
- The application image builder and deployer roles can work independently without the need to jointly build a single app image.
- Because init containers run to completion before any app containers start, init containers offer a mechanism to block or delay app container startup until a set of preconditions are met. Once preconditions are met, all of the app containers in a Pod can start in parallel.

Examples

- Wait for a Service to be created, using a shell one-line command like:

  $for i in {1..100}; do sleep 1; if nslookup myservice; then exit 0; fi; done; exit 1

- Register this Pod with a remote server from the downward API with a command like:

  $curl -X POST http://$MANAGEMENT_SERVICE_HOST:$MANAGEMENT_SERVICE_PORT/register -d 'instance=$(<POD_NAME>)&ip=$(<POD_IP>)'

- Wait for some time before starting the app container with a command like

  $ sleep 60

Services

- A Kubernetes Service is an abstraction that defines a logical set of Pods and a policy by which to access them.
- Key Points:
  - Exposing pods to external network
  - Enables decoupling between internal microservices.
  - Provides a stable IP and DNS name for a set of Pods.
  - Allows load-balancing traffic across multiple Pods.

Types of Services

- ClusterIP:
  - Default service type.
  - Exposes the Service on a cluster-internal IP.
- NodePort:
  - Exposes the Service on each Node's IP at a static port.
  - Use Case: Ideal for exposing a service to the external world.
- LoadBalancer:
  - Creates an external load balancer in the cloud (if supported).
  - Use Case: Distributes traffic to multiple nodes.
- ExternalName:
  - Maps the Service to the contents of the externalName field.
  - Use Case: Allows using a service external to the cluster.

Labels and selectores

- Labels are key/value pairs that are attached to objects, such as pods.
- Labels are intended to be used to specify identifying attributes of objects that are meaningful and relevant to users, but do not directly imply semantics to the core system.

```
"metadata": {
 "labels": {
  "key1" : "value1",
  "key2" : "value2"
 }
}
```

Example labels

"release" : "stable", "release" : "canary"

"environment" : "dev", "environment" : "qa", "environment" : "production"

"tier" : "frontend", "tier" : "backend", "tier" : "cache"

"partition" : "customerA", "partition" : "customerB"

"track" : "daily", "track" : "weekly"

Label selectors

Unlike names and UIDs, labels do not provide uniqueness. In general, we expect many objects to carry the same label(s).

Via a label selector, the client/user can identify a set of objects. The label selector is the core grouping primitive in Kubernetes.

The API currently supports two types of selectors: equality-based and set-based. A label selector can be made of multiple requirements which are comma-separated. In the case of multiple requirements, all must be satisfied so the comma separator acts as a logical AND (&&) operator.

Replication in Kubernetes

A process in Kubernetes to maintain a consistent number of pod replicas.
It ensures high availability of applications, load distribution, and enables scalability.

ReplicationController

ReplicaSet

ReplicationController

A ReplicationController is a feature in Kubernetes that ensures a specified number of pod replicas are running at all times.

It's the original form of replication in Kubernetes but is now considered legacy. However, understanding it is important for comprehending Kubernetes' evolution.

ReplicaSet

A ReplicaSet is an improved version of the ReplicationController. It aims to maintain a stable set of replica Pods running at any given time.

It supports set-based selector requirements, which is more expressive than the simple equality-based selectors used by ReplicationControllers.

Feature Comparison:

Selector Support: ReplicaSet supports set-based selectors, while ReplicationController only supports equality-based selectors.

Flexibility: ReplicaSet allows more flexibility in managing pods.

Recommended Use: ReplicaSet is recommended in modern Kubernetes applications.

Deployment

- Kubernetes Deployments are high-level constructs that manage the deployment and scaling of a set of Pods.
- They provide declarative updates to applications.
- Deployments manage the creation, deletion, and updating of Pods, ensuring that a specified number of Pods are running at all times.
- Ideal for stateless applications.

Key Features of Deployments

- Automated Rollouts & Rollbacks: Deployments automatically update Pods and their configurations in a controlled fashion (e.g., rolling updates).
- Scaling: Deployments can be scaled up or down easily.
- Self-healing: Automatically replaces failed, unresponsive, or unhealthy Pods.
- Versioning and History: Maintains the history of updates and allows rollback to previous states.

Deployment Workflow

- Define Deployment: Declare the desired state in a Deployment configuration file (YAML).
- Apply Deployment: Apply the configuration using kubectl apply.
- Update Pods: Deployment creates/updates Pods in a rolling update fashion by default.
- Monitor Rollout Status: Track the status of a Deployment update.
- Rollback if Necessary: Revert to a previous Deployment state if issues arise.

Deployment Strategies

- Rolling Update (Default): Update Pods in a rolling fashion, minimizing downtime. Ideal for production.
- Recreate: All existing Pods are killed before new ones are created. Suitable for stateful applications where concurrent versions should not run.

Rolling Out Deployment Parameters

- maxUnavailable: Specifies the maximum number of Pods that can be unavailable during the update process.
- maxSurge: Specifies the maximum number of Pods that can be created over the desired number of Pods.
- minReadySeconds: Specifies the minimum number of seconds for which a newly created Pod should be ready without any of its containers crashing, for it to be considered available.

Setting Hardware Limits

- Prevents a single pod from consuming excessive resources, which can impact other pods.
- Helps Kubernetes scheduler in placing pods on nodes efficiently.
- Ensures applications perform as expected under resource constraints.

Request and Limit

- Resource Requests: The amount of CPU and memory Kubernetes guarantees for a container. If a container exceeds its request for a resource, it might be throttled back down to its request level if that resource is needed elsewhere.
- Resource Limits: The maximum amount of CPU and memory that a container is allowed to use. If a container exceeds its limit, it might be terminated and possibly restarted, depending on its restart policy.

Kubernetes Namespaces

- Namespaces in Kubernetes are a way to divide cluster resources between multiple users.
- They are used to provide a scope for names.
- Resource names need to be unique within a namespace, but not across namespaces.
- Commonly used for multi-tenant environments.

Default Namespaces

- default: The default namespace for objects with no other namespace.
- kube-system: The namespace for objects created by the Kubernetes system.
- kube-public: This namespace is mostly reserved for cluster usage, where the resources should be visible and readable publicly throughout the whole cluster.
- Kube-node-lease: It is used to track node heartbeats more efficiently, which improves the performance and scalability of large Kubernetes clusters.

Namespaces Effects

Feature	Namespaces Affect It?	**Notes**
Pod-to-pod communication	❌ No	All pods can talk to each other by default
DNS service discovery	✅ Yes	Resolution is namespace-scoped
Network isolation	❌ Not by default	Use NetworkPolicies to enforce isolation
Service accessibility across namespace	✅ With FQDN	Requires fully qualified service name
Logging/Monitoring segmentation	✅ Yes	Easier by grouping traffic/logs per namespace

Kubernetes Resource Scheduling

Resource scheduling in Kubernetes involves selecting the most suitable node for a pod based on resource requirements and constraints.

The scheduler evaluates each node's available resources (CPU, memory, etc.) and schedules pods where the requirements can be met.

Resource Requests and Limits

- Resource Requests and Limits:
- Requests are used by the scheduler to select an appropriate node.
- Limits prevent a pod from using more than a specified amount of a resource.

Node Affinity

- Node affinity allows you to constrain pods to nodes with specific labels.

```
nodeSelectorTerms:
    - matchExpressions:
      - key: <label-key>
        operator: In
        values:
        - <label-value>
```

levels of strictness

- preferredDuringSchedulingIgnoredDuringExecution :
  - the scheduler tries to enforce, but not guaranteed
- RequiredDuringSchedulingIgnoredDuringExecution :
  - must be enforced

Taints and Tolerations

- Taints are applied to nodes and pods with matching tolerations are allowed to schedule onto these nodes.
  - Taint a node: kubectl taint nodes <node-name> key=value:taint-effect
  - Add tolerations to your pod/deployment YAML:

```
spec:
  tolerations:
  - key: "key"
    operator:  "Equal"
    value: "value"
    effect: "NoSchedule"
```

Types of Taint Effects

- NoSchedule: Pods without the toleration are not scheduled on the node.
- PreferNoSchedule: The system tries to avoid placing a pod that does not tolerate the taint on the node but is not guaranteed.
- NoExecute: Pods without the toleration are evicted from the node if they are already running on the node, and are not scheduled on it in the future.

Key Differences

Aspect	Node Affinity/Anti-Affinity	Taints and Tolerations
Application	Property of pods to get scheduled on appropriate nodes.	Properties of nodes to prevent certain pods from being scheduled unless they have a matching toleration.
Control Aspect	About attracting or repelling pods from nodes.	About allowing only certain pods on a node.
Scheduling Decisions	Based on labels on nodes.	Based on special markers on nodes that must be explicitly tolerated by pods.

Config Map

- ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.
- They store non-confidential data in key-value pairs, which can be consumed by pods or used to set environment variables.
- Storing configuration files, command-line arguments, environment variables, port numbers, etc.
- Useful for creating easily modifiable configurations without the need for rebuilding the container image.

Volumes

Volume in Kubernetes

- A volume in Kubernetes is a storage unit attached to Pods, which outlives the lifecycle of a container within the Pod.
- It allows data to persist and be shared between containers of a Pod.

Types of Volumes

- emptyDir: A simple empty directory used for storing transient data. Its lifecycle is tied to the Pod.
- hostPath: Used for mounting directories from the host node's filesystem into a Pod.
- PersistentVolume (PV) and PersistentVolumeClaim (PVC): Provide a way to use external storage systems (like cloud storage or NFS) and manage their lifecycle independently from Pods.

Persistent Volume Workflow

- Create Persistent Volume
- Create Persistent Volume Claim
- Create pod and assign claim

Access Modes

- ReadWriteOnce (RWO)
  - The volume can be mounted as read-write by a single node.
- ReadOnlyMany (ROX)
  - The volume can be mounted as read-only by many nodes.
- ReadWriteMany (RWX)
  - The volume can be mounted as read-write by many nodes.

DaemonSet

- a DaemonSet is a resource type that ensures a copy of a pod runs on all (or some) nodes in the cluster.
- When nodes are added to the cluster, pods are added to them automatically by the DaemonSet. Similarly, when nodes are removed from the cluster, those pods are garbage collected.

DaemonSet Use Cases

- Running a cluster storage daemon on every node, such as glusterd or ceph.
- Running a logs collection daemon on every node, such as fluentd or logstash.
- Running a node monitoring daemon on every node, like Prometheus Node Exporter, collectd, Datadog agent, or New Relic agent.

Networking

Basics of Kubernetes Networking

- Pod-to-Pod Communication: Pods can communicate with each other across nodes without using NAT (Network Address Translation).
- No IP Overlap: Each pod gets a unique IP address, ensuring no IP overlap in the cluster.
- Service Discovery: Kubernetes uses services for communication between different pods, providing a stable endpoint for pod-to-pod communication.

Types of Services in Kubernetes

- ClusterIP: Exposes the service on an internal IP in the cluster, making it reachable only within the cluster.
- NodePort: Exposes the service on each Node's IP at a static port.
- LoadBalancer: Exposes the service externally using the cloud provider's load balancer.
- ExternalName: Maps the service to an externalName field by returning a CNAME record.

Understanding Network Policies

- Network Policies in Kubernetes are used to control the flow of traffic between pod groups.
- Network policies are implemented by the network plugin in the cluster, so you need a network solution that supports NetworkPolicy.
- They use labels to select pods and define rules which specify what traffic is allowed to and from these pods.

NetworkPolicy Examples

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-frontend
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
        role: frontend
```

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-namespace
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: dev-namespace
```

Ingress Controller

- Ingress is a Kubernetes API object that provides HTTP and HTTPS routing to services.
- An Ingress Controller is a daemon that runs within a Kubernetes cluster and implements the rules defined in Ingress resources.
- It acts as a reverse proxy and load balancer, providing a single point of entry for external traffic.

Common Ingress Controllers

- NGINX Ingress Controller
- Traefik
- HAProxy
- Istio Gateway

Nginx : https://github.com/kubernetes/ingress-nginx

Configuration

- Create Deployment / Service / Pod
- Expose with service
- Create Ingress Controller over service to manage traffic

Configuration

- Create Deployment / Service / Pod
- Expose with service
- Create Ingress Controller over service to manage traffic

Routing Options

- Host-Based Routing:
  - outes traffic based on the HTTP host header (i.e., the domain name).
- Path-Based Routing
  - Routes traffic based on the URL path.
- Combination of Host and Path-Based Routing
  - Combines both host and path-based routing.
- TLS/SSL Routing
  - Routes traffic based on TLS/SSL certificates (usually with SNI).

Monitoring

Key Metrics to Monitor in Kubernetes

- **Cluster Health**
  - Monitor the status and health of nodes, pods, and control plane components.
  - Key metrics include node availability, pod status (Running, Pending, Failed), and component statuses.
- **Resource Utilization**
  - Track CPU and memory usage across nodes and pods.
  - Monitor disk I/O and network bandwidth utilization.
- **Workload-Specific Metrics**
  - Track application-specific metrics, such as request rates, error rates, and response times for services.

Monitoring Tools for Kubernetes

- Prometheus
- Grafana
- Elastic Stack (ELK)
- Kubernetes Dashboard
- Third-Party Tools
  - Tools like Datadog, New Relic, and Dynatrace provide advanced monitoring capabilities with Kubernetes integration.

Log Monitoring

- Loging Specific Services
  - Pod, Service, Deployment, etc.
- Aggregate Logs
  - Set up a log aggregation system using Fluentd, Fluent Bit, or Logstash.
- Integrate with ELK Stack or a Centralized Logging Solution
  - Forward logs to Elasticsearch for indexing and use Kibana for visualization.

kibana

554 hits

New Save Open Share Inspect Auto-refresh Last 15 minutes

>_ Search... (e.g. status:200 AND extension:PHP) Options Refresh

Discover
Visualize
Dashboard
Timelion
Canvas
Machine Learning
Infrastructure
Logs
APM
Dev Tools
Monitoring
Management

kubernetes.pod_name: "node" Add a filter + Actions

Edit filter

Filter Edit Query DSL

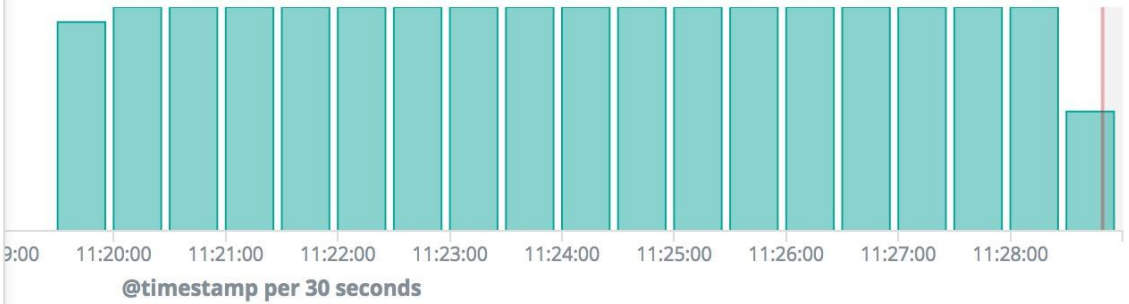
kubernetes.pod_name is node

Label

Optional

Cancel Save

:13:47.893 - January 21st 2019, 11:28:47.893 — Auto



t _id
t _index
_score
t _type
t docker.container_id
t kubernetes.container...
t kubernetes.container...
t kubernetes.container...
t kubernetes.host
t kubernetes.labels.app
t kubernetes.labels.po...

January 21st 2019, 11:20:35.102 kubernetes.pod_name: node-795bc66469-4wwgj log: 2019-01-21 18:20:35.102 INFO it works stream: stdout docker.container_id: b5619dc39548b217eb6346bf87f414eb11543482e2e84f776abe1229a233db2f kubernetes.container_name: node kubernetes.namespace_name: default kubernetes.container_image: fluentd-node-sample:latest kubernetes.container_image_id: docker://sha256:6e80a40146f450056fc54f9255552d1519e2ebd66415d727e4a9d0b09df23985 kubernetes.pod_id: 17e4af96-1da9-11e9-b7c8-f2deaba1f648

January 21st 2019, 11:20:31.094 kubernetes.pod_name: node-795bc66469-4wwgj log: 2019-01-21 18:20:31.094 INFO it works stream: stdout docker.container_id: b5619dc39548b217eb6346bf87f414eb11543482e2e84f776abe1229a233db2f kubernetes.container_name: node kubernetes.namespace_name: default kubernetes.container_image: fluentd-node-sample:latest kubernetes.container_image_id: docker://sha256:6e80a40146f450056fc54f9255552d1519e2ebd66415d727e4a9d0b09df23985 kubernetes.pod_id: 17e4af96-1da9-11e9-b7c8-f2deaba1f648

January 21st 2019, 11:20:34.100 kubernetes.pod_name: node-795bc66469-4wwgj log: 2019-01-21 18:20:34.100 INFO it works stream: stdout docker.container_id: b5619dc39548b217eb6346bf87f414eb11543482e2e84f776abe1229a233db2f kubernetes.container_name: node kubernetes.namespace_name: default kubernetes.container_image: fluentd-node-sample:latest kubernetes.container_image_id: docker://sha256:6e80a40146f450056fc54f9255552d1519e2ebd66415d727e4a9d0b09df23985 kubernetes.pod_id: 17e4af96-1da9-11e9-b7c8-f2deaba1f648

D Default
Collapse

Security

Kubernetes Security Best Practices 1

- **Cluster Access Control**
  - Implement Role-Based Access Control (RBAC) to restrict access to Kubernetes API.
  - Use service accounts for applications running on Kubernetes and limit their permissions using RBAC.
- **Pod Security Policies**
  - Use Pod Security Policies (PSPs) to control the security specifications a pod must adhere to.
  - Enforce policies like disallowing privileged containers, enforcing read-only root file systems, and requiring the use of volume types.

Example: Role for the Namespace

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: my-namespace
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

Example: Bind With User

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: my-namespace
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

More Examples

apiGroups: ["apps"]
resources: ["deployments"]
verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]

apiGroups: [""]
resources: ["configmaps"]
verbs: ["get", "list", "create", "update", "delete"]

More Examples Contd.

```
apiGroups: ["*"]
resources: ["*"]
verbs: ["*"]
```

```
apiGroups: ["*"]
resources: ["*"]
verbs: ["get", "list", "watch"]
```

Verbs List

Verb	Description
get	Retrieve an individual resource.
list	Retrieve a list of resources.
watch	Watch for changes to a resource.
create	Create a new instance of a resource.
update	Update an existing resource.
patch	Partially update an existing resource.
delete	Delete a resource.
deletecollection	Delete a collection of resources.
exec	Execute a command in a container.
bind	Grant a Role or ClusterRole to a subject.
escalate	Allow a subject to request higher privileges.
use	Use a resource, typically applicable to PodSecurityPolicies.

Kubernetes Security Best Practices 2

- **Network Policies**
  - Define network policies to control the communication between pods.
  - Use them to isolate workloads and control ingress and egress traffic.
- **Securing the Kubelet**
  - Protect the Kubelet, the primary node agent, by using Kubelet authentication and authorization.

Kubernetes Security Best Practices 3

- **Image Security**
  - Use trusted base images.
  - Scan images for vulnerabilities and use image signing to maintain integrity.
- **Secrets Management**
  - Store sensitive data like passwords and tokens using Kubernetes Secrets.
  - Encrypt secrets at rest and manage access to secrets using RBAC.

Troubleshooting

Basic Troubleshooting Steps

- **Check Cluster Health**
  - Kubectl get nodes
- **Inspect Pod Status and Logs**
  - Kubectl get pods
- **Describe Resources**
  - Kubectl describe <pod/node/service/> name
- **Check logs**
  - kubectl logs <pod-name>

Common Issues and Solutions

- **Pod Scheduling Failures**
  - Inspect events in kubectl describe pod for reasons like insufficient resources, taints, or affinity/anti-affinity rules.
- **Image Errors**
  - Issues like ImagePullBackOff or ErrImagePull often indicate problems with the container image (e.g., wrong image name, private registry authentication).
- **Crashing or Unhealthy Pods**
  - Review pod logs and check for application-level errors.

Common Issues and Solutions

- **Service Discovery and Networking Issues**
  - Ensure services are correctly defined and selectors match pod labels.
  - Check network policies for restrictive rules blocking traffic.

- **Persistent Volume Issues**
  - For PVs not binding to PVCs, ensure the access modes and storage requirements match.
  - Check the storage class and provisioner configurations

Advanced Troubleshooting Tools

- **Kubernetes Dashboard**
  - Provides a GUI for easier resource management and observation.
- **Debugging Pods**
  - Create a debug pod or use kubectl exec to run diagnostic commands within a cluster context.
- **Monitoring and Logging Tools**
  - Use Prometheus, Grafana, ELK Stack, or similar tools to monitor cluster metrics and logs.

Containers on Cloud

- Containers on the cloud refer to the deployment of containerized applications in a cloud computing environment.
- Popular cloud providers offering services for containerized applications include:
  - Amazon Web Services (AWS) with Amazon ECS and Amazon EKS,
  - Microsoft Azure with Azure Kubernetes Service (AKS)
  - Google Cloud Platform (GCP) with Google Kubernetes Engine (GKE)

ECS

- Amazon Elastic Container Service (ECS) is a fully managed container orchestration service provided by Amazon Web Services (AWS).
- ECS simplifies the deployment, management, and scaling of containerized applications using Docker containers. It is a native AWS service designed to work seamlessly with other AWS resources, making it a popular choice for running containers in the AWS cloud.

EKS

- Amazon Elastic Kubernetes Service (EKS) is a fully managed Kubernetes service provided by Amazon Web Services (AWS). EKS simplifies the deployment, management, and scaling of containerized applications using Kubernetes, an open-source container orchestration platform.
- With EKS, users can run Kubernetes applications on AWS without the need to manage the underlying Kubernetes control plane.

Elastic Container Registry (ECR)

- Amazon Elastic Container Registry (ECR) is a fully managed Docker container registry service provided by Amazon Web Services (AWS).
- ECR makes it easy to store, manage, and deploy Docker container images, allowing developers to build, store, and deploy their containerized applications on AWS.

Thank you

Shubham Dalvi

shubham.dalvi@rpsconsulting.in

www.rpsconsulting.in